

Cahier des charges

Projet développeurs : API & simulateur

1. Contexte du projet

Les étudiants créent l'interface de zéro, ils **consomment une API fournie par l'enseignant** pour construire une app.

L'objectif est de vous mettre dans une situation réaliste de développement front-end : travailler à partir d'une documentation d'API, gérer des appels asynchrones, structurer un état applicatif complexe, et restituer des données sous forme de graphiques clairs et interactifs.

2. Objectifs pédagogiques

- Lire et exploiter une documentation d'API (endpoints, paramètres, structure de réponse)
- Effectuer des appels API asynchrones (fetch/axios) et gérer les états de chargement/erreur
- Structurer une application front avec une gestion d'état adaptée
- Manipuler une bibliothèque de data visualization (Chart.js, Recharts, D3.js, etc.)
- Concevoir une interface de simulation claire malgré la complexité des données (plusieurs véhicules, plusieurs variables)
- Travailler la robustesse : validation des entrées utilisateur, gestion des cas limites (valeurs manquantes, erreurs API)

3. Description fonctionnelle du produit final

L'utilisateur doit pouvoir :

1. Sélectionner un ou plusieurs véhicules à comparer (modèle, motorisation)
2. Renseigner des paramètres personnels : kilométrage annuel estimé, durée de possession prévue (en années), éventuellement sa région/pays (impact carburant/électricité)
3. Lancer une simulation et obtenir, pour chaque véhicule :
 - Coût total sur la durée choisie
 - Répartition détaillée : carburant/électricité, entretien, assurance, décote estimée
 - Coût moyen par mois et par kilomètre

4. Spécification de l'API fournie

Cette section décrit ce que l'API expose. Les étudiants n'ont pas à la développer, uniquement à la consommer.

4.1 Endpoints

GET /vehicules Retourne la liste des véhicules disponibles pour la simulation.

Exemple de réponse :

```
[
  {
    "id": "veh_001",
    "marque": "Renault",
    "modele": "Clio",
    "motorisation": "essence",
    "prix_achat": 21000,
    "consommation_moyenne": 5.5,
    "annee": 2025
  },
  {
    "id": "veh_002",
    "marque": "Renault",
    "modele": "Megane E-Tech",
    "motorisation": "electrique",
    "prix_achat": 35000,
```

```
"consommation_moyenne": 16.5,  
"annee": 2025  
}  
]
```

POST /simulation Calcule le coût total de possession pour un ou plusieurs véhicules selon les paramètres fournis.

Corps de la requête attendu :

```
{  
  "vehicule_ids": ["veh_001", "veh_002"],  
  "kilometrage_annuel": 15000,  
  "duree_annees": 5,  
  "region": "FR"  
}
```

Exemple de réponse :

```
{  
  "resultats": [  
    {  
      "vehicule_id": "veh_001",  
      "cout_total": 28450,  
      "cout_mensuel_moyen": 474,  
      "cout_par_km": 0.38,  
      "detail": {  
        "carburant": 6200,  
        "entretien": 3100,  
        "assurance": 5200,  
        "decote_estimee": 8500,  
        "autres": 5450  
      },  
      "evolution_annuelle": [  
        {"annee": 1, "cout_cumule": 9200},  
        {"annee": 2, "cout_cumule": 14800},  
        {"annee": 3, "cout_cumule": 19900},  
        {"annee": 4, "cout_cumule": 24500},  
        {"annee": 5, "cout_cumule": 28450}      ]  
    }  
  ]  
}
```

```
]
}
]
}
```

GET /vehicules/:id Retourne le détail d'un véhicule (utile pour afficher une fiche complémentaire).

4.2 Codes d'erreur à gérer côté front

Code	Signification	Comportement attendu côté front
400	Paramètres invalides (ex. kilométrage négatif)	Afficher un message d'erreur clair, ne pas lancer la simulation
404	Véhicule introuvable	Retirer le véhicule de la sélection avec avertissement
500	Erreur serveur	Afficher un message générique et proposer de réessayer

5. Attentes côté front-end

5.1 Fonctionnalités indispensables (MVP)

Fonctionnalité	Description
Sélection de véhicules	Interface permettant
Appel API et gestion des états	Indicateur de chargement pendant l'appel, gestion des erreurs (voir 4.2)
Graphique de répartition des coûts	Un graphique (barres empilées ou équivalent) par véhicule, montrant la répartition carburant/entretien/assurance/décote

Graphique d'évolution du coût cumulé	Un graphique en courbes montrant l'évolution du coût cumulé sur la durée choisie, pour comparer plusieurs véhicules sur le même graphique
Tableau récapitulatif	Vue tabulaire avec coût total, coût mensuel moyen, coût par km, pour chaque véhicule sélectionné
Mise à jour dynamique	La modification d'un paramètre relance la simulation sans recharger la page

5.2 Fonctionnalités bonus

Fonctionnalité	Description
Export des résultats	Génération d'un PDF ou d'une image du comparatif
Persistance locale	Sauvegarde de la dernière simulation dans le navigateur (localStorage)
Recommandation automatique	Mise en avant du véhicule le plus économique selon les critères choisis
Accessibilité des graphiques	Alternative textuelle/tabulaire aux graphiques pour les lecteurs d'écran

6. Contraintes techniques

- Langage front: React
- Bibliothèque de graphiques au choix (Chart.js, Recharts, D3.js, ApexCharts...), en cohérence avec le langage choisi
- Les appels à l'API doivent être asynchrones et ne jamais bloquer l'interface
- Prévoir un fichier `.env` ou équivalent pour l'URL de l'API (pas d'URL codée en dur dispersée dans le code)

7. Livrables attendus

1. **Code source complet**, hébergé sur un dépôt *Git* avec README expliquant comment lancer le projet
2. **Démo fonctionnelle** (sur vercel)
3. **Documentation courte** : choix techniques, structure du projet, difficultés rencontrées, gestion des erreurs mise en place

Lien : <https://carapi.app/api>